



Universidad de
los Andes



**FACULTAD
DE INGENIERÍA
Y CIENCIAS
APLICADAS**

PROYECTO DE DESARROLLO DE SOFTWARE

Proyecto 3: Web+Embebidos

Carrera

Computer Science Engineering

Profesor

Matías Recabarren Bahamondes

Alumnos

Vicente Cuevas L.

Camila Hinojosa A.

Ignacio Andrés Liberon S.

Manuel José Tagle A.

Santiago de Chile, 10 de octubre del 2024

1. ARQUITECTURA DE SOFTWARE.....	3
1. Sistema de Administración (Django):.....	3
2. Controlador de Casilleros (ESP32).....	4
2. CONTROLADOR DE CASILLEROS.....	5
A. Microcontrolador ESP32:.....	5
B. Cámara.....	5
C. Motor Solenoide.....	5
D. Fuente de energía autónoma.....	5
E. Breadboard.....	5
F. LEDs.....	5
G. Sensor magnético.....	5
H. Pantalla.....	5
2.1 Modelo de identificación de signos.....	8
2.2 Comunicación con el Sistema de Administración.....	9
3. SISTEMA DE ADMINISTRACIÓN.....	10
3.1 Arquitectura del sistema:.....	10
A. Frontend.....	10
a. Frameworks y librerías principales.....	10
i. React.....	10
ii. Material-UI (MUI).....	10
iii. Axios.....	10
B. Backend.....	11
a. Frameworks y librerías principales.....	11
i. Django.....	11
ii. Django Rest Framework (DRF).....	11
iii. PostgreSQL.....	11
C. Reconocimiento de Gestos.....	11
a. Frameworks y librerías principales:.....	11
i. MediaPipe.....	11
ii. OpenCV.....	11
iii. TensorFlow.....	11
3.2 Comunicación con los Controladores.....	12
3.2.1 Sincronización de Contraseñas y Configuración.....	12
3.2.2 Notificaciones de Apertura/Cierre.....	12
3.2.2 Actualización de Modelos de Reconocimiento de Gestos.....	12
3.3 Wireframes de las vistas principales.....	13
3.3.1 Wireframes de los usuarios.....	13
3.3.2 Wireframes de los Casilleros y Controladores.....	14

1. ARQUITECTURA DE SOFTWARE

La arquitectura del sistema está compuesta por dos softwares principales: el **Sistema de Administración**, desarrollado en Django, y los **Controladores de Casilleros**, que se basan en el microcontrolador ESP32, equipado con una cámara para el reconocimiento de gestos y un motor solenoide para el mecanismo de apertura/cierre de los casilleros. A continuación, se describe cada software y se identifican sus componentes internos y externos de cada parte del sistema:

- 1. Sistema de Administración:** Este software es una aplicación web que permite la gestión de los casilleros, el registro de usuarios y la asignación de claves y dueños a cada casillero. Además, ofrece un dashboard para visualizar métricas de uso. Todo esto a través del uso de Django como framework web, PostgreSQL como base de datos y una API REST para la comunicación entre el ESP32 y la aplicación web.

Componentes Internos:

- Framework Web (Django)
- Base de Datos (PostgreSQL)
- API REST

Componentes Externos:

- Servidor Web
- Proveedor de Correo Electrónico

2. Controlador de Casilleros (ESP32): El ESP32 actúa como el componente de hardware que gestiona el acceso a los casilleros. El sistema de bloqueo de los casilleros utiliza un motor solenoide controlado mediante el software del ESP32. Además, el controlador de los casilleros está equipado con una cámara que reconoce gestos, los cuales son interpretados por el software del ESP32 para determinar si la clave es correcta o no, permitiendo la apertura del casillero.

Componentes Internos:

- Microcontrolador (ESP32)
- Cámara
- Motor Solenoide
- Firmware del ESP32

Componentes Externos:

- Broker MQTT
- Red Wi-Fi

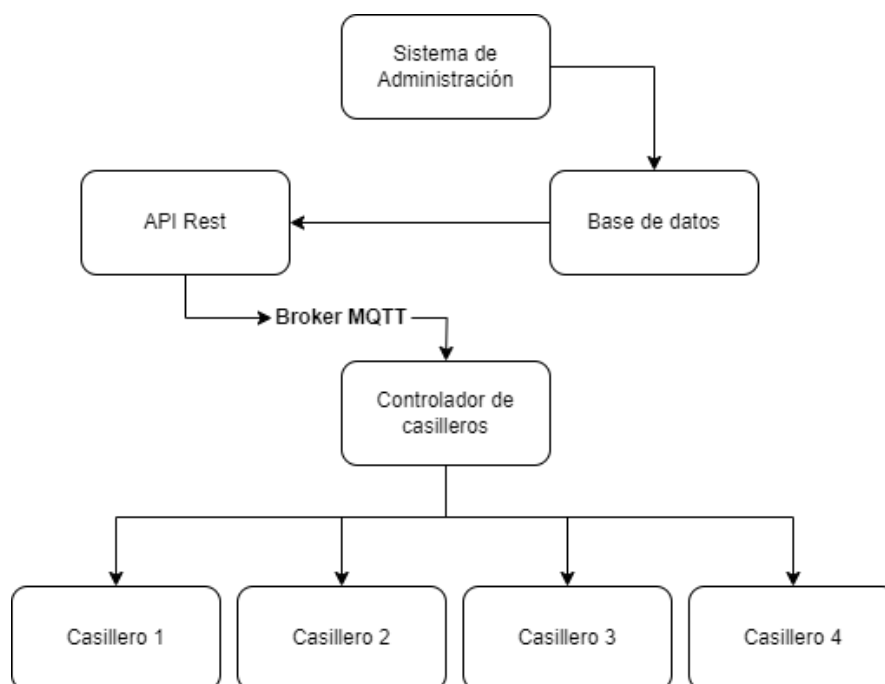


Figura 1.1: Diagrama Arquitectura de Software

2. CONTROLADOR DE CASILLEROS

El controlador de casilleros está compuesto por los siguientes componentes mecánicos y sensores, que permiten un funcionamiento eficiente para la apertura y cierre de los casilleros:

- A. Microcontrolador ESP32:** El ESP32 es el encargado de gestionar toda la lógica de acceso a los casilleros, recibir la información capturada por la cámara y activar el mecanismo de cierre.
- B. Cámara:** El controlador está equipado con una cámara que captura las señas manuales realizadas por el usuario para el ingreso de la clave. Estas señas se comparan con los símbolos almacenados en el sistema para determinar si se permite la apertura del casillero.
- C. Motor Solenoide:** El solenoide actúa como un sistema de cerradura electrónica que se activa al recibir la señal correspondiente del ESP32, permitiendo la apertura o el cierre del casillero.
- D. Fuente de energía autónoma:** El sistema funcionará con baterías recargables para garantizar su independencia de un suministro de energía externa.
- E. Breadboard:** El breadboard será el encargado de unir todos los cables que conectan los componentes físicos con el ESP32.
- F. LEDs:** Luces LED que dan feedback al usuario sobre el estado del candado Abierto/Cerrado y sobre si la cámara está recibiendo bien la información.
- G. Sensor magnético:** Sensor que verifica si la puerta del casillero está abierta o cerrada para poder entregarle esa información al Sistema de Administración.
- H. Pantalla:** Una pantalla que muestra lo que ve la cámara para que el usuario pueda ver si está captando los gestos bien.
- I. Selector electromecánico:** Un selector conectado al microcontrolador para determinar la que casillero será el que se abrirá a continuación.

La interacción entre el usuario y el controlador sigue el siguiente orden:

1. El usuario se acerca al controlador y selecciona el número de casilleros que desea abrir mediante el selector electromecánico.
2. Tras ver la clave en el sistema web, el usuario presiona el botón de activación, lo que enciende la cámara y los LEDs para indicar que está lista para capturar los gestos.
3. Realiza la secuencia de gestos frente a la cámara, que son procesados por el ESP32.
4. Tras esto, el ESP32 procesa los datos de la cámara y compara los gestos capturados con la clave almacenada para el casillero seleccionado.
5. Si los gestos coinciden con la clave, el ESP32 activa el motor solenoide para abrir el casillero.
6. Cuando el casillero se abre, el sensor magnético detecta el cambio de estado y envía esta información al ESP32.
7. El ESP32 notifica al sistema de administración sobre el evento de apertura.
8. El usuario puede cerrar el casillero presionando nuevamente el botón de cierre, lo que activa el ciclo de cierre.

A continuación, se detallan dos diagramas en los cuales se aprecia el mecanismo de apertura y cerrado de cada casillero y el detector de señas, los cuales llamaremos casillero y controlador respectivamente:

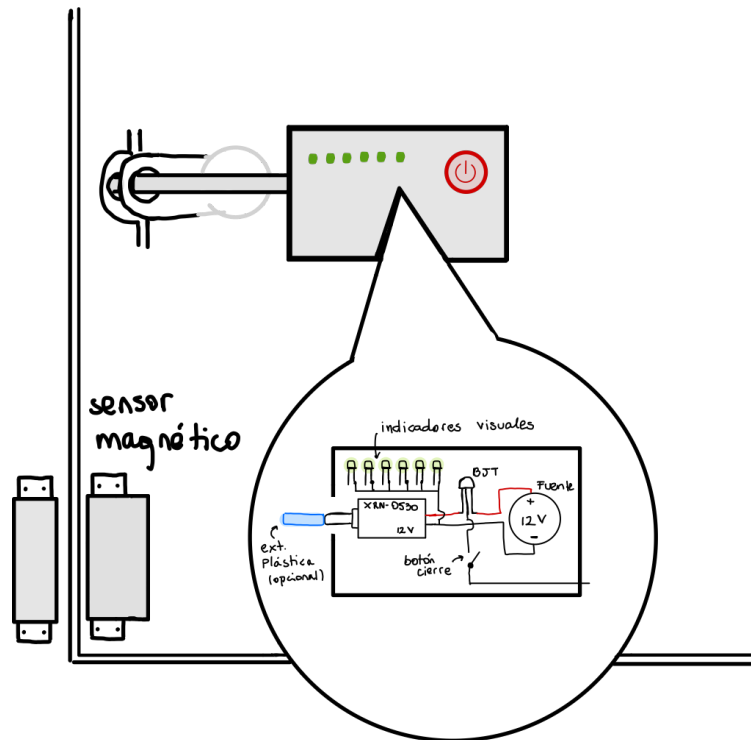


Figura 2.1: Mecanismo de apertura y cierre de casillero
Fuente: Elaboración propia

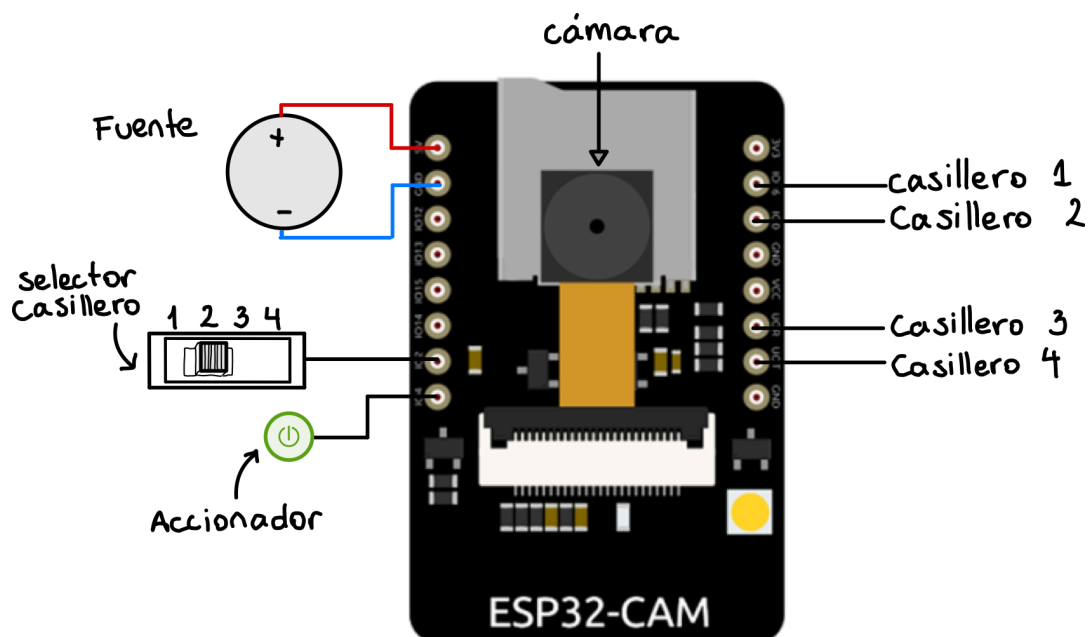


Figura 2.2: Mecanismo del controlador de casilleros
Fuente: Elaboración propia

2.1 Modelo de identificación de signos

El **modelo de identificación** de símbolos utilizado en este proyecto está basado en el repositorio [Hand Gesture Recognition](#) de kinivi, que implementa un sistema de reconocimiento de gestos manuales utilizando MediaPipe y Python. Este proyecto emplea el pipeline de MediaPipe Hands para detectar y extraer los puntos clave de la mano, transformándolos en una serie de coordenadas. Estas coordenadas son posteriormente clasificadas mediante un modelo de Machine Learning que identifica el gesto realizado por el usuario.

En nuestro proyecto de casilleros, este modelo se adapta y optimiza para funcionar en un dispositivo embebido como el ESP32-CAM. El flujo de trabajo implica reducir la complejidad del modelo utilizando técnicas como cuantización y poda, permitiendo que el procesamiento de los gestos sea eficiente y de bajo consumo de recursos en el dispositivo. Asimismo, la estructura del modelo se modifica para poder integrarse en el sistema de control de apertura/cierre, capturando los gestos necesarios para validar el acceso a los casilleros.

Esta integración permite una identificación robusta y precisa de gestos en tiempo real, garantizando la seguridad y la funcionalidad requerida para el sistema de control de casilleros.

2.2 Comunicación con el Sistema de Administración

La comunicación entre el Controlador de Casilleros y el Sistema de Administración se establece mediante el protocolo MQTT. Este protocolo permite el intercambio de mensajes en tiempo real a través de un broker MQTT, facilitando la sincronización de información entre ambos componentes. Esta arquitectura permite que los controladores envíen notificaciones al sistema de administración sobre eventos de apertura y cierre, y que el sistema de administración pueda enviar actualizaciones de configuración o modelos de reconocimiento de gestos a los controladores de forma remota.

En particular, la actualización remota de contraseñas en el controlador se gestiona a través de un canal dedicado de MQTT, lo cual permite mantener los datos de autenticación actualizados en todo momento. Cuando el administrador cambia o asigna una nueva contraseña a un casillero específico desde la plataforma de administración, el sistema publica este cambio en un tópico MQTT diseñado para la actualización de contraseñas. El ESP32, que actúa como el controlador de los casilleros y está suscrito a este tópico, recibe inmediatamente la nueva clave y la almacena en su memoria interna, reemplazando la contraseña anterior.

Gracias a este proceso, cualquier intento de acceso al casillero será validado con la clave más reciente sin necesidad de intervención física en el dispositivo. Esto garantiza una autenticación precisa y segura, y también elimina la necesidad de actualizaciones manuales o intervención directa en el hardware. La arquitectura basada en MQTT asegura una gestión eficiente de la comunicación entre el controlador y el sistema de administración, permitiendo que tanto las configuraciones de seguridad como los modelos de reconocimiento de gestos se mantengan actualizados y alineados con las políticas de administración establecidas.

3. SISTEMA DE ADMINISTRACIÓN

El **Sistema de Administración** será el encargado de gestionar el correcto funcionamiento del controlador de los casilleros. Este sistema será una plataforma web que sea accesible desde cualquier dispositivo, tanto computadoras como dispositivos móviles.

3.1 Arquitectura del sistema:

El sistema seguirá una arquitectura del tipo “cliente-servidor”. Este se compondrá de un Frontend y un Backend:

A. Frontend: Proporciona la interfaz del usuario.

a. Frameworks y librerías principales:

- i. **React:** Es una librería de JavaScript usada para la creación de interfaces de usuario dinámicas y reactivas. Permitirá a los administradores gestionar los casilleros y visualizar la información importante en tiempo real.
- ii. **Material-UI (MUI):** Librería que se usará para la construcción de interfaz visual, permitiendo un diseño funcional y atractivo.
- iii. **Axios:** Librería que manejará la comunicación entre Frontend y Backend mediante solicitudes HTTP.

B. Backend: Gestiona la lógica interna de la aplicación, la autenticación y la comunicación con los controladores del proyecto.

a. Frameworks y librerías principales:

- i. **Django:** Framework de Python que se encargará de la lógica y autenticaciones de la aplicación.
- ii. **Django Rest Framework (DRF):** kit de herramientas para la construcción de APIs web en Django. Proporciona una forma sencilla y rápida de crear APIs RESTful.
- iii. **PostgreSQL:** Base de datos relacional para el manejo de consultas SQL y controladores.

C. Reconocimiento de Gestos: Se utilizará el repositorio de GitHub [hand-gesture-recognition-mediapipe](https://github.com/mediapipe/hand-gesture-recognition-mediapipe). Este repositorio facilitará la implementación del sistema de reconocimiento de gestos manuales que permitirá la interacción de usuarios con los casilleros.

a. Frameworks y librerías principales:

- i. **MediaPipe:** Framework de Google que permite la detección y el reconocimiento de manos y gestos. Es esencial para identificar las posiciones de las manos y mapear los gestos manuales.
- ii. **OpenCV:** Librería de visión artificial usada para el procesamiento de imágenes en tiempo real. Se usará para capturar y procesar las manos del usuario.
- iii. **TensorFlow:** Librería utilizada para la implementación de modelos de Aprendizaje de Máquinas (Machine Learning).

3.2 Comunicación con los Controladores

La comunicación entre el Sistema de Administración y los Controladores de Casilleros se realiza a través del protocolo MQTT. Este protocolo de mensajería ligera es ideal para entornos IoT, permitiendo el intercambio de datos en tiempo real mediante un modelo de **publicador-suscriptor**.

3.2.1 Sincronización de Contraseñas y Configuración: Cada vez que el administrador actualiza o cambia una contraseña o configuración de un casillero, el Sistema de Administración publica esta información en un canal MQTT específico para actualizaciones (por ejemplo, `/casilleros/actualizacion_claves`). El controlador de casilleros, suscrito a este canal, recibe el mensaje y actualiza sus contraseñas y configuraciones internas en tiempo real.

3.2.2 Notificaciones de Apertura/Cierre: Cuando un usuario realiza un gesto para abrir o cerrar un casillero, el controlador envía un mensaje al Sistema de Administración en un tópico MQTT para notificaciones de eventos (por ejemplo, `/casilleros/evento`). Esta notificación incluye detalles como el ID del casillero y el estado de apertura/cierre, lo que permite al administrador monitorear en tiempo real las actividades en cada casillero.

3.2.2 Actualización de Modelos de Reconocimiento de Gestos: El modelo de reconocimiento de gestos puede ser actualizado remotamente por el Sistema de Administración. Cuando se dispone de una nueva versión del modelo, el sistema lo publica en un canal MQTT específico para actualizaciones de modelos (por ejemplo, `/casilleros/actualizacion_modelo`). Los controladores, suscritos a este canal, descargan el nuevo modelo y lo integran, asegurando que el sistema utilice siempre la versión más reciente.

3.3 Wireframes de las vistas principales

A continuación se presentarán versiones tempranas de la interfaz de usuario de la plataforma. En esta sección, se presentarán los datos importantes a mostrar y como se organizarán.

3.3.1 Wireframes de los usuarios

Las siguientes imágenes muestran los wireframes para el dashboard de los usuario y superusuarios:



Figura 3.1: Wireframe Dashboard Usuario - Fuente: Elaboración propia



Figura 3.2: Wireframe Dashboard - Fuente: Elaboración propia

3.3.2 Wireframes de los Casilleros y Controladores

En la siguiente vista de muestra un Index de Controladores, bajo cada controlador, se pueden ver los casilleros asociados y sus detalles:

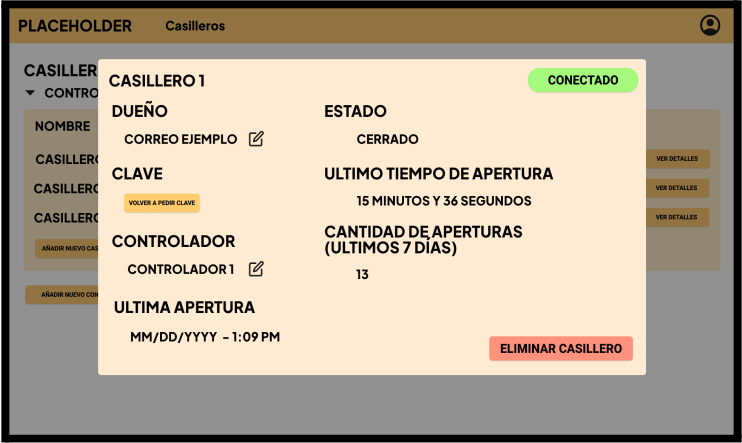
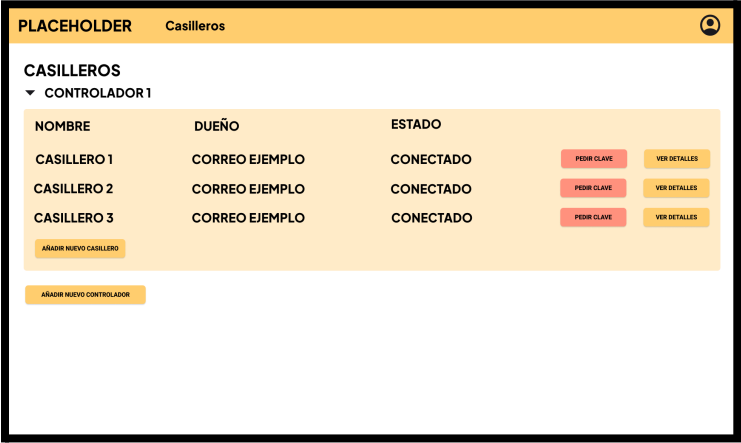


Figura 3.3: Wireframe Index Casilleros; Figura 3.4: Wireframe Detalle de Casillero

Fuente: Elaboración propia

En esta pestaña también se pueden añadir casilleros y controladores nuevos:

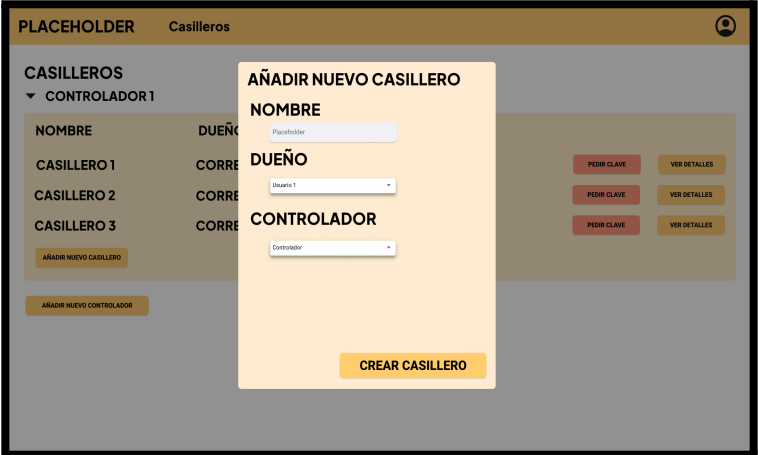
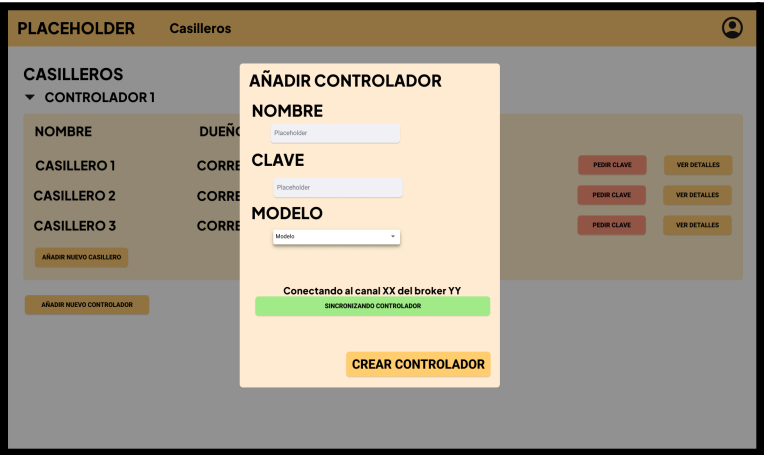


Figura 3.5: Wireframe Crear Controlador; Figura 3.6: Wireframe Crear Casillero

Fuente: Elaboración propia